

Deterministic quasipolynomial-time mean-payoff games

OpenAI

September 25, 2026

Abstract

We give a deterministic algorithm that computes the complete zero-threshold winning set of a finite mean-payoff game with arbitrary signed integer edge weights encoded in binary. For total explicit input length L , it uses $2^{O((\log(L+2))^2)}$ bit operations. A reduction also computes the exact rational value at every vertex and globally optimal positional strategies for both players within the same quasipolynomial bound.

Contents

1	Introduction	2
2	From potential comparisons to a labelling problem	5
3	Moving witnesses to a narrower box	8
4	A deterministic recursion on width and mass	10
5	Why the labels satisfy every witness	12
6	Counting calls and bit operations	15
7	Exact values and optimal positional strategies	18
8	Fixed- and minimum-credit energy games	20
9	Tropical feasibility and fixed corank	23

1 Introduction

In a mean-payoff game, two players choose edges of a finite graph and compete over the long-run average weight of the resulting infinite walk. The objective is numerical, but the question of who can enforce a nonnegative average is a finite decision problem. A central algorithmic issue is the representation of the weights: a procedure polynomial in their absolute values may take exponential time in the length of the input. We give a deterministic algorithm whose running time is quasipolynomial in that binary length and which determines all winning starting vertices together.

1.1 The problem and the main theorem

Let $G = (V, E)$ be a finite directed graph with $V = \{1, \dots, n\}$ and $n \geq 1$. Each vertex has at least one outgoing edge and belongs to exactly one of V_{Max} and V_{Min} . The edge weights $w : E \rightarrow \mathbb{Z}$ are arbitrary signed binary integers. Self-loops and parallel edges are allowed; parallel edges are separate entries of the explicit edge list. At a vertex its owner chooses the next edge. Both players' strategies may depend on the entire finite history. For an infinite play $\pi = (e_0, e_1, \dots)$, write

$$\text{MP}_w(\pi) = \liminf_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} w(e_t).$$

We count the complete input length L in a fixed conventional binary encoding of the graph, endpoints, ownership, and weights. For a start v and strategies σ, τ , denote their induced play by $\pi_{v, \sigma, \tau}$.

Theorem 1.1. *A uniform deterministic algorithm returns exactly the set*

$$U = \{v \in V : \exists \text{ Max strategy } \sigma \ \forall \text{ Min strategies } \tau, \text{MP}_w(\pi_{v, \sigma, \tau}) \geq 0\}$$

for every game specified above. For an absolute constant $C > 0$, the complete computation uses at most

$$2^{C(\log_2(L+2))^2}$$

bit operations in a standard model polynomially equivalent to a uniform deterministic Turing machine. The bound includes preprocessing, exact arithmetic, recursion, and output of the entire set U .

A positional strategy chooses its outgoing edge using only the current vertex. Section 7 derives the exact rational value at every starting vertex and globally optimal positional strategies for both players, within the same quasipolynomial bit bound. Section 8 decides whether a prescribed initial credit can keep every finite accumulated weight nonnegative in a one-resource energy game, and computes minimum winning credits. Section 9 applies classical reductions to tropical feasibility and fixed-corank tropical-rank tests. A polynomial bound for the complete binary input would be a further improvement.

1.2 Earlier algorithms and binary weights

Ehrenfeucht and Mycielski proved the existence of positional optimal strategies for mean-payoff games [13]. Positional determinacy places the threshold problem in $\text{NP} \cap \text{coNP}$. Zwick and Paterson developed pseudopolynomial algorithms and complexity reductions [30]. Such bounds

can be polynomial in the number of vertices and the largest absolute weight while remaining exponential in the binary encoding length. This distinction is central to Theorem 1.1.

The energy formulation asks whether a finite initial credit can keep all finite accumulated weights nonnegative. Brim, Chaloupka, Doyen, Gentilini, and Raskin developed bounded integer progress measures and monotone lifting, with an $O(|E|nW_0)$ threshold and winning-region bound when W_0 bounds the absolute weights [7, Theorem 8]. Dorfman, Kaplan, and Zwick developed a deterministic exponential-time approach based on accelerated potential updates and scaling; Austin and Dell’Erba corrected its update procedure [11, 3]. Kozachinskiy gives a polyhedral interpretation and a weight-independent $n^{O(1)}2^{n/2}$ arithmetic bound for energy winners [19].

Strategy-improvement algorithms provide another important comparison. Björklund and Vorobyov proved an expected $2^{O(\sqrt{n \log n})}$ arithmetic bound for the threshold partition on graphs without parallel edges [5, Theorem 7.1], following work with Sandberg [4]. Recent analyses concern Random-Action-Removal [29] and Switch-All and Random-Edge for energy games [12]. Iteration counts and arithmetic bounds must be distinguished from bit bounds: strategy evaluation and the sizes of intermediate numbers contribute to the latter.

Parity games use an extremal priority occurring infinitely often to determine the winner. They reduce in polynomial time to mean-payoff threshold games using weights of exponential magnitude but polynomial binary length [16, Section 3]. Calude, Jain, Khoussainov, Li, and Stephan gave a quasipolynomial algorithm for parity games [9]. The reduction goes from parity to mean payoff, so this result does not by itself give a quasipolynomial algorithm for arbitrary binary-weight mean-payoff games. Daviaud, Jurdziński, and Lazić combined succinct parity progress measures with energy bounds in a pseudo-quasipolynomial algorithm for mean-payoff parity games [10]; that bound still contains a numerical weight parameter. Smoothed polynomial results for independent Gaussian perturbations of payoffs on ergodic game graphs concern a different input model [22].

1.3 How the proof works

An integer potential assigns a number to each vertex. First replace $w(e)$ by $\hat{w}(e) = (n+1)w(e) + 1$, which makes every simple cycle sum nonzero while separating negative original sums from nonnegative ones. For $z \in \mathbb{Z}^V$, let $F_i(z)$ be the maximum of $\hat{w}(e) + z_j$ over edges $e : i \rightarrow j$ at a Max vertex, and the minimum at a Min vertex. For integer vectors $A \leq u$, the box $[A, u]$ consists of all integer vectors between them, coordinatewise. In this box, a subsolution is a vector $x \in [A, u]$ satisfying $x_i \leq F_i(x)$ wherever $x_i > A_i$; a supersolution is a vector $y \in [A, u]$ satisfying $y_i \geq F_i(y)$ wherever $y_i < u_i$. Clipping a vector to the box means replacing coordinate z_i by $\min\{u_i, \max\{A_i, z_i\}\}$. The comparison theorem in *Randomized quasipolynomial-time mean-payoff games*, our companion article, gives a unique fixed point h of $z \mapsto \min\{u, \max\{A, F(z)\}\}$, and the inequalities $x \leq h \leq y$ for every such pair. These finite-box facts and their basic iteration procedure are the only results imported from the companion in the proof of Theorem 1.1 [24, Definition 2.2 and Lemmas 2.3–2.4]. Section 2 states that interface in full.

The same section places each coordinate of h near one of the two boundaries of a sufficiently wide box $[0, D]^V$. The upper band is winning for Max and the lower band for Min. Iterating the clipped operator from the bottom can require a number of steps proportional to the numerical width D . We instead ask which boundary labels are forced, without computing h .

For a box $[A, A+D]$, give the vertices positive rational masses a_i and b_i . For a vertex set S , its masses are $a(S) = \sum_{i \in S} a_i$ and $b(S) = \sum_{i \in S} b_i$. A Max witness is a subsolution whose support $\{i : x_i > A_i\}$ has a -mass at most one; it requires a positive label at coordinates near the upper boundary. A Min witness is the dual supersolution with support $\{i : y_i < A_i + D\}$ of b -mass

at most one; it requires a negative label near the lower boundary. The task is to meet every requirement of every witness at once. With initial masses $1/n$, the box solution qualifies on both sides, so these requirements determine every vertex’s winner.

To solve that task recursively, we move an integer pivot vector k near the middle of the box. For a Max witness, a common translation aligns the largest deviation $x_i - k_i$ with the top of a narrower box centered at k . Clipping at the bottom leaves only coordinates whose deviations are close to that largest value. A dual operation handles Min witnesses. Section 3 proves that these operations preserve the required inequalities and restrict supports to subsets of the original supports.

The algorithm in Section 4 first moves k downward, then upward. Narrow recursive calls choose which coordinates move. Each pass protects the extremal deviation of all witnesses of one type. When a pass stops, any still-needed witness of the other type must have a set of coordinates close to an extremum with a -mass greater than $3/4$ for a Max witness or b -mass greater than $3/4$ for a Min witness. Otherwise the last recursive call would have forced another movement. This use of the final, stationary call is the key deterministic certificate.

One call on a half-width box labels those large-mass sets. Its labels need not settle the original boundary requirements. They instead specify new masses: coordinates labelled in a witness’s favor become lighter, and the others heavier. Section 5 shows that every unsettled witness still has support mass at most one. Meanwhile, every product $a_i b_i$ increases by a fixed factor. Once every product exceeds one, each coordinate is excluded from at least one type of witness support, and the labels can be chosen directly. Starting with masses $1/n$, only $O(\log(n+1))$ such product increases can therefore occur along a branch. The final call on the original box uses the reweighted masses to settle the remaining requirements. Every child except the half-width call increases the products; that exceptional child instead reduces the width. Section 6 counts the possible positions of the product increases along a branch and bounds the exact arithmetic in each call, giving the stated bit complexity.

1.4 Potential methods and recursive precision

Gurvich, Karzanov, and Khachiyan developed potential transformations in their algorithm for cyclic games [15, 18]: adding differences of endpoint potentials to edge weights preserves cycle sums while changing the local inequalities. Paired potentials appear in the work of Lifshits and Pavlov [21]. Cadilhac, Casares, and Ohlmann give a general framework for accelerated value iteration [8]. These methods and the progress-measure approach described above provide the setting for our finite-box inequalities.

The closest recursive ancestry is Parys’s reduced-precision parity algorithm [25, Algorithm 2, Section 5, and Lemma 6.1] and its development by Lehtinen, Parys, Schewe, and Wojtczak [20]. Their guarantees protect all dominions—sets within which one player can keep play and win—within specified size bounds, and their recursion separates one child retaining full precision from children of reduced precision. We likewise use simultaneous obligations for small objects, but smallness here means rational mass of a potential’s support. Translation near an extremal deviation, the stationary-pivot certificates, and reweighting establish the needed progress for that notion of smallness.

Ohlmann’s symmetric mean-payoff recursion supplies closely related potential-based context [23]; its subexponential time analysis is left open in the cited version. Jurdziński, Morvan, Ohlmann, and Thejaswini use two simultaneous labellings for symmetric acceleration of parity algorithms [17]; a related development by Thejaswini, Ohlmann, and Jurdziński retains both decompositions

to accelerate symmetric recursion [26]. Arnold, Niwiński, and Parys study recursive interval restriction in a quasipolynomial black-box algorithm for nested fixed-point evaluation [2]. Their bound is expressed in Boolean-lattice dimension and fixed-point nesting depth. The integer box here has chains of length $nD + 1$, so that result does not directly give the dependence on $\log D$ established by our translation and mass arguments.

Fijalkow, Gawrychowski, and Ohlmann studied mean-payoff value iteration using universal graphs and lower bounds for those representations [14]. Those lower bounds constrain universal-graph representations. The present recursion constructs and reweights boxes for the given instance without using such a representation.

The companion algorithm [24] returns potential vectors with probabilistic comparison guarantees stated separately for each fixed admissible vector. Here the recursive guarantee is simultaneous over all qualifying witnesses and concerns their boundary labels. The two algorithms share the finite-box theory stated in Section 2; the companion’s randomized proof is independent of the deterministic recursion. Section 7 also explains how its winning-set interface yields randomized exact values and positional strategies.

2 From potential comparisons to a labelling problem

This section specifies the finite-box input used from the companion, then proves the game reduction at the width required by our recursion. The outcome is a labelling problem whose obligations are expressed by potential vectors. Later sections solve that problem without finding the potential that certifies the winning partition.

2.1 The common finite-box interface

Fix the input graph and ownership throughout. Following the integer cycle perturbation of Björklund and Vorobyov [5, Section 9], put

$$\hat{w}(e) = (n + 1)w(e) + 1, \quad W = \max_{e \in E} |\hat{w}(e)|. \quad (2.1)$$

The edge set is nonempty. Each transformed weight is a nonzero integer, so $W \geq 1$. If a simple directed cycle has length ℓ and original sum c , then

$$\hat{w}(C) = (n + 1)c + \ell, \quad 1 \leq \ell \leq n. \quad (2.2)$$

For $c \geq 0$ the transformed sum is positive; for $c \leq -1$ it is negative. Thus no simple cycle has zero transformed sum, and positive transformed sums correspond exactly to nonnegative original sums. A self-loop has $\ell = 1$, and the calculation applies to every actual edge sequence when parallel edges are present.

For all $z \in \mathbb{Z}^V$, define

$$F_i(z) = \begin{cases} \max_{e: i \rightarrow j} (\hat{w}(e) + z_j), & i \in V_{\text{Max}}, \\ \min_{e: i \rightarrow j} (\hat{w}(e) + z_j), & i \in V_{\text{Min}}. \end{cases} \quad (2.3)$$

The finite, nonempty outgoing-edge sets ensure that these extrema are attained. With coordinate-wise vector order and common scalar shifts,

$$z \leq z' \implies F(z) \leq F(z'), \quad F(z + t) = F(z) + t \quad (t \in \mathbb{Z}). \quad (2.4)$$

For integer vectors $A \leq u$, the notation $[A, u]$ denotes all integer vectors between them. A *subsolution* is a full vector $x \in [A, u]$ satisfying $x_i \leq F_i(x)$ at every coordinate where $x_i > A_i$. A *supersolution* is a full vector $y \in [A, u]$ satisfying $y_i \geq F_i(y)$ wherever $y_i < u_i$. No inequality is required on the respective excluded boundary. These conventions agree with the companion's Definition 2.2 [24].

Lemma 2.1 (Finite-box comparison and basic iteration). *For the operator in Equation (2.3) and arbitrary finite integer bounds $A \leq u$, every subsolution x and supersolution y in $[A, u]$ satisfy $x_i \leq y_i$ for all $i \in V$. There is a unique $h \in [A, u]$ that is both a subsolution and a supersolution. It is precisely the unique fixed point of*

$$T_{A,u}(z) = \min\{u, \max\{A, F(z)\}\}.$$

Consequently every such pair satisfies $x \leq h \leq y$.

The following synchronous iteration finds h . Start with $p = A$; evaluate $q = T_{A,u}(p)$; return p if $q = p$, and otherwise set $p = q$ and repeat. It uses at most

$$1 + \sum_{i \in V} (u_i - A_i)$$

evaluations of $T_{A,u}$, including the final stability test.

We use this statement from the complete September 25 companion, Lemmas 2.3–2.4 [24]. It permits negative endpoints and collapsed coordinates $A_i = u_i$. In particular its comparison conclusion concerns all vertices, even though each hypothesis is imposed only on its stated support. The operator and integer hypotheses are exactly those above, and Equation (2.2) verifies the required absence of zero simple-cycle sums. The sandwich inequality follows by comparing with h on each side. We call h the *box solution* and the iteration **Basic**. The algorithm will invoke **Basic** only when the box width is bounded by an absolute constant.

2.2 The two bands of a wide box

Choose the initial dyadic width

$$D_0 = \min\{2^d : d \in \mathbb{Z}_{\geq 0}, 2^d \geq 64(n+1)(W+1)\}. \quad (2.5)$$

The next proposition establishes the game reduction for this width. The companion uses width $8nW$ in its own reduction; that different choice is not an input to the proof here.

Proposition 2.2 (Winning partition from the wide box). *Let h be the box solution in $[0, D_0]^V$. Then*

$$V_{l_0} = \{i : h_i \leq nW\}, \quad V_{h_i} = \{i : h_i \geq D_0 - nW\}$$

partition V . There is a positional Max strategy under which every play starting in V_{h_i} has original mean payoff at least zero. There is a positional Min strategy under which every play starting in V_{l_0} has original upper limiting average at most $-1/n$. Both assertions allow arbitrary history-dependent opposition.

Proof. We first locate the coordinates of h , then construct the two strategies. If $0 < h_i < D_0$, both box inequalities apply and give $h_i = F_i(h)$. Follow an edge attaining that equality and continue from its head as long as the current coordinate is interior. A first repeated vertex would yield a

simple cycle of tight equalities $h_i = \hat{w}(e) + h_j$, with transformed sum zero. This is impossible by Equation (2.2). A boundary is therefore reached within n edges. Each edge changes the potential by at most W . Including the length-zero path from a boundary coordinate, every coordinate is within nW of either 0 or D_0 . Since $D_0 > 2nW$, the two bands are disjoint and exhaust V .

Every high coordinate is above the lower boundary. At a high Max vertex choose one outgoing edge satisfying

$$h_i \leq \hat{w}(e) + h_j; \quad (2.6)$$

such an edge exists by the subsolution inequality. At a high Min vertex every outgoing edge satisfies this inequality. Each of these permitted edges has

$$h_j \geq D_0 - (n+1)W > nW,$$

so its head is high by the partition just proved. Fixing the Max choices and extending them arbitrarily outside the high band gives a positional strategy keeping all consistent plays from high vertices in that band.

On a simple cycle of permitted edges, Equation (2.6) telescopes to a nonnegative transformed sum. Its original sum is therefore nonnegative. Let $W_0 = \max_e |w(e)|$. Delete simple cycles successively from any finite play prefix until fewer than n edges remain. The removed cycles have nonnegative original sums, and the remaining walk has weight at least $-(n-1)W_0$. Hence every prefix of every consistent play has at least that weight. Dividing by its length and taking a lower limit proves Max's assertion.

For the other band, every low coordinate is below the upper boundary. At a low Min vertex fix an edge attaining the minimum in $F_i(h)$, and at a low Max vertex permit all outgoing edges. The supersolution inequality gives

$$h_i \geq \hat{w}(e) + h_j \quad (2.7)$$

for each permitted edge. Its head satisfies $h_j \leq (n+1)W < D_0 - nW$, so it is low. These choices, extended arbitrarily elsewhere, give a positional Min strategy preserving the low band.

Every simple cycle of permitted edges now has nonpositive transformed sum and hence original integer sum at most -1 . Deleting simple cycles from a prefix of length T leaves at most $n-1$ edges. Each deleted cycle has at most n edges, so the number removed is at least $(T - (n-1))/n$. The original prefix weight is at most

$$-\frac{T - (n-1)}{n} + (n-1)W_0.$$

Its upper limiting average is at most $-1/n$. The prefix estimates on both sides apply to every consistent edge sequence, independently of how the opposing player uses the history. $\square$

2.3 Simultaneous boundary obligations

The recursion works on inputs

$$A \in \mathbb{Z}^V, \quad D = 2^d \ (d \in \mathbb{Z}_{\geq 0}), \quad a, b \in \mathbb{Q}_{>0}^V. \quad (2.8)$$

Its box is $[A, u]$ with $u = A + D$, and its claim margin is $P = D/32$. For $S \subseteq V$ let $a(S) = \sum_{i \in S} a_i$ and $b(S) = \sum_{i \in S} b_i$. The positive rational vectors a, b are called *masses*.

Definition 2.3 (Witnesses and their claims). A *Max witness* for (A, D, a, b) is a subsolution $x \in [A, u]$ with support $S_x = \{i : x_i > A_i\}$ and $a(S_x) \leq 1$. It *claims* each i with $x_i \geq u_i - P$. A *Min witness* is a supersolution $y \in [A, u]$ with support $S_y = \{i : y_i < u_i\}$ and $b(S_y) \leq 1$. It claims each i with $y_i \leq A_i + P$. A witness is *relevant* when it claims at least one index.

Since $P < D$, every claim belongs to its witness's support. Claims of opposite types cannot coincide: otherwise box comparison would give $u_i - P \leq x_i \leq y_i \leq A_i + P$, contradicting $2P < D$. Masses limit the witnesses covered by an obligation; the graph itself always retains every vertex and edge.

Theorem 2.4 (Simultaneous labelling). *For every input in Equation (2.8), the procedure $\text{Label}(A, D, a, b)$ of Section 4 terminates and returns a vector in $\{+, -\}^V$. It assigns $+$ to every index claimed by any Max witness and $-$ to every index claimed by any Min witness.*

All witnesses of a call are covered by this one output. They are neither enumerated nor supplied to the procedure. On vertices with no claim the theorem permits either label.

Apply this guarantee to

$$A = 0, \quad D = D_0, \quad a_i = b_i = 1/n. \quad (2.9)$$

Every support then has mass at most one. The box solution h is therefore a witness of both types. Moreover $P = D_0/32 \geq nW$, so its Max claims include all high vertices and its Min claims include all low vertices. By Proposition 2.2, a labelling satisfying Theorem 2.4 returns the exact winning set by taking its positively labelled vertices. This uses the existence and boundary location of h , without executing Basic on the wide initial box.

3 Moving witnesses to a narrower box

A recursive child will receive a box centered at a pivot k near the middle of its parent box. To transfer a parent's obligations to this child, we translate a witness by one scalar and clip it at a boundary. This section establishes the exact support and claims of the translated vector. It is the geometric part of the argument and uses only monotonicity, common-shift invariance, and the witness definition.

Consider an input (A, D, a, b) with dyadic $D \geq 4096$. In addition to $u = A + D$ and $P = D/32$, fix

$$c = A + D/2, \quad M = D/8, \quad q = D/128, \quad p = q/32 = D/4096. \quad (3.1)$$

The vector c is the center, M bounds the pivot's displacement, q is the narrow child width, and p will be one pivot step. All these quantities, as well as $q/2$ and $D/4$, are integral. We consider integer pivots $c - M \leq k \leq c + M$.

For a relevant Max witness x , let

$$s_x(k) = \max_{i \in V} (x_i - k_i), \quad g_i^x(k) = s_x(k) - (x_i - k_i). \quad (3.2)$$

We call $s_x(k)$ its *peak* and $g_i^x(k)$ its *gap* at i . For a relevant Min witness y the dual definitions are

$$s_y(k) = \max_{i \in V} (k_i - y_i), \quad g_i^y(k) = s_y(k) - (k_i - y_i). \quad (3.3)$$

Every gap is a nonnegative integer. The following lemma treats the two widths that the recursion will use.

Lemma 3.1 (Translated supports and claims). *Let (A, D, a, b) be an input with $D \geq 4096$, let $c - M \leq k \leq c + M$ be integral, and let $r \in \{q, D/2\}$. For any relevant Max witness x , the vector*

$$x'_i = \max\{k_i - r/2, x_i - s_x(k) + r/2\} \quad (3.4)$$

is an integer subsolution in $[k - r/2, k + r/2]$. Its support is

$$S_{x'} = \{i : g_i^x(k) < r\} \subseteq S_x.$$

For any positive new Max masses a' with $a'(S_{x'}) \leq 1$, it is a Max witness and claims every i with $g_i^x(k) \leq r/32$.

For any relevant Min witness y , the vector

$$y'_i = \min\{k_i + r/2, y_i + s_y(k) - r/2\} \quad (3.5)$$

is an integer supersolution in the same box, with support

$$S_{y'} = \{i : g_i^y(k) < r\} \subseteq S_y.$$

For any positive new Min masses b' with $b'(S_{y'}) \leq 1$, it is a Min witness and claims every i with $g_i^y(k) \leq r/32$. In particular, the corresponding unchanged masses a and b always make these translated vectors witnesses.

Proof. The support assertion requires knowing which coordinates could survive the clipping. Relevance supplies that information. A Max claim at j gives $x_j \geq A_j + D - P$, while $k_j \leq A_j + D/2 + M$; hence

$$s_x(k) \geq D/2 - M - P. \quad (3.6)$$

If $i \notin S_x$, then $x_i = A_i$ and $x_i - k_i \leq -D/2 + M$. Combining the two inequalities gives

$$g_i^x(k) \geq D - 2M - P = 23D/32 > D/2. \quad (3.7)$$

The untruncated term in Equation (3.4) is

$$k_i + r/2 - g_i^x(k).$$

It lies below or at the new upper boundary. Taking its maximum with the lower boundary puts x' inside the new box, with support exactly where $g_i^x(k) < r$. Since $r \leq D/2$, Equation (3.7) shows that this support lies in S_x . Integrality follows from the integral half-widths and peaks.

Write $t = -s_x(k) + r/2$. Globally $x' \geq x + t$; on its new support, $x'_i = x_i + t$ and the parent's subsolution inequality applies. Equation (2.4) therefore yields

$$F_i(x') \geq F_i(x + t) = F_i(x) + t \geq x_i + t = x'_i.$$

Thus x' is a subsolution. Its witness condition is precisely the stated mass bound. If $g_i^x(k) \leq r/32$, its untruncated entry is at least $k_i + r/2 - r/32$, the new Max claim threshold; taking a maximum with the lower boundary cannot destroy that claim.

For Min, relevance likewise gives $s_y(k) \geq D/2 - M - P$. Outside S_y , one has $y_i = A_i + D$, so $k_i - y_i \leq -D/2 + M$. The gap is again at least $23D/32 > D/2$. Now the translated term before clipping is

$$k_i - r/2 + g_i^y(k).$$

It is above or at the lower boundary, and taking its minimum with the upper boundary leaves support exactly at gaps $< r$. The support is contained in S_y by the same outside-support estimate. Set $t = s_y(k) - r/2$. The vector inequality $y' \leq y + t$ and equality $y'_i = y_i + t$ on the new support imply

$$F_i(y') \leq F_i(y + t) = F_i(y) + t \leq y_i + t = y'_i.$$

This is the required supersolution inequality. The mass condition makes y' a witness, and a gap at most $r/32$ puts it at most $r/32$ above the new lower boundary, giving the asserted claim. Finally, support containment and positivity imply $a(S_{x'}) \leq a(S_x) \leq 1$ and $b(S_{y'}) \leq b(S_y) \leq 1$ for unchanged masses. $\square$

The two thresholds serve different purposes: gaps strictly below r describe the surviving support, whereas gaps at most $r/32$ give claims.

4 A deterministic recursion on width and mass

We now give the procedure whose output meets the simultaneous obligations of Theorem 2.4. Every step uses exact integers or rationals and label vectors from children. The witnesses from the preceding section are proof objects; no execution searches for or tests them.

For an integer pivot k and an even width r , a *centered call* with masses (a', b') means $\text{Label}(k - r/2, r, a', b')$, on the box $[k - r/2, k + r/2]$. This notation will only be used with integer half-widths.

In each pivot pass, one side's masses remain unchanged. Its translated witnesses force labels at all gaps at most $p = q/32$; those labels will keep these coordinates fixed while the pivot moves. The later width- $D/2$ call has unchanged masses on both sides and makes all gaps below q into claims, supplying the labels for reweighting.

Procedure 4.1 ($\text{Label}(A, D, a, b)$). Set $u = A + D$ and $P = D/32$. Execute the first applicable base case; if neither applies, proceed through Steps 3–6.

1. **Mass base case.** If $a_i b_i > 1$ for all $i \in V$, return $+$ where $b_i > 1$ and return $-$ at the remaining vertices.
2. **Width base case.** If $D < 4096$, compute the box solution z by **Basic**. Return $+$ at i if $z_i > A_i + D/2$, and $-$ otherwise.
3. **Downward pass.** Use c, M, q, p from Equation (3.1) and initialize $k = c$. Repeatedly make a centered width- q call with masses $(a, \frac{4}{3}b)$ and, from its labels, set simultaneously

$$k_i^{\text{new}} = \begin{cases} \max\{c_i - M, k_i - p\}, & \text{if the label at } i \text{ is } -, \\ k_i, & \text{if the label at } i \text{ is } +. \end{cases}$$

Replace k by k^{new} . Stop when the update leaves k unchanged, denote this last pivot by $k^\downarrow$, and record

$$I_- = \{i : k_i^\downarrow < c_i - P\}.$$

4. **Upward pass.** Start from $k = k^\downarrow$. Repeatedly make a centered width- q call with masses $(\frac{4}{3}a, b)$ and set simultaneously

$$k_i^{\text{new}} = \begin{cases} \min\{c_i + M, k_i + p\}, & \text{if the label at } i \text{ is } +, \\ k_i, & \text{if the label at } i \text{ is } -. \end{cases}$$

Replace k by k^{new} . Stop when it is unchanged, write k^f for the final pivot, and record

$$I_+ = \{i : k_i^f > c_i + P\}.$$

5. Provisional labels and new masses. Make one centered width- $D/2$ call at k^f with masses (a, b) . For its returned vector $\lambda^0 \in \{+, -\}^V$, define

$$(\bar{a}_i, \bar{b}_i) = \begin{cases} (\frac{4}{5}a_i, \frac{8}{5}b_i), & \lambda_i^0 = +, \\ (\frac{8}{5}a_i, \frac{4}{5}b_i), & \lambda_i^0 = -. \end{cases} \quad (4.1)$$

6. Continuation and overrides. Call $\text{Label}(A, D, \bar{a}, \bar{b})$. In that child's output put $-$ on I_- and $+$ on $I_+ \setminus I_-$, leaving every other label unchanged. Return the resulting vector.

Each pass includes a final child call whose update changes no coordinate. Correctness will use that call's guarantee together with the stationary update. The two passes run in the stated order; within one update every coordinate uses the same returned label vector and the same old pivot.

4.1 Integral nested boxes and bounded passes

The next facts depend only on the mechanics of the procedure, not on the meaning of the returned labels.

Lemma 4.2. *Every recursive child is a valid input of the form Equation (2.8), and its box is contained in its parent's box. In a nonbase call the pivot satisfies*

$$\begin{aligned} c - M \leq k \leq c & \quad \text{during the downward pass,} \\ c - M \leq k \leq c + M & \quad \text{during the upward pass.} \end{aligned}$$

If child calls terminate, each pass makes at most $1024n + 1$ calls.

Proof. In the recursive case, $D \geq 4096$ is dyadic, so all parameters in Equation (3.1), both child half-widths, and every pivot coordinate are integral. The child widths $D/128$ and $D/2$ are positive powers of two. The update caps imply the stated pivot bounds. A centered child has width $r \leq D/2$, and its endpoints satisfy

$$c - M - r/2 \geq A + D/8 \geq A, \quad c + M + r/2 \leq A + 7D/8 \leq A + D.$$

The continuation has the unchanged parent box. All mass multipliers preserve positivity and rationality.

The pivot lies on the grid of spacing p through c , since $M/p = 512$ is integral and both caps belong to that grid. In a changing update at least one coordinate moves by at least p . Each pass is coordinatewise monotone over a range of at most $2M$, so a coordinate moves at most $2M/p = 1024$ times. At most $1024n$ updates can change the pivot. One further child call certifies the unchanged update at which the pass stops. $\square$

4.2 A rank that decreases for every child

For positive masses define

$$\mu(a, b) = \min_{i \in V} a_i b_i, \quad \rho = \frac{32}{25}, \quad B(\mu) = \begin{cases} 0, & \mu > 1, \\ 1 + \lfloor \log_\rho(1/\mu) \rfloor, & 0 < \mu \leq 1. \end{cases} \quad (4.2)$$

The integer B measures how many guaranteed increases of the minimum mass product can precede the mass base case. It is used for analysis only; the executable procedure does not evaluate logarithms.

Lemma 4.3 (Well-founded recursion). *The procedure terminates on every input in Equation (2.8). Writing $D = 2^d$, every child of a nonbase call has smaller rank*

$$R = d + B(\mu(a, b)).$$

The provisional child in Step 5 is the only child that preserves B . Every other child decreases B by at least one.

Proof. If $\mu > 1$, the mass base case applies. Otherwise multiplying μ by a factor at least ρ decreases B by at least one: either the new value exceeds one, or its logarithm in Equation (4.2) falls by at least one before rounding down.

In either pivot pass, each product $a_i b_i$ is multiplied by $4/3 \geq \rho$. In the continuation, each product is multiplied by $(4/5)(8/5) = \rho$, independent of the provisional label. Thus these children decrease B and do not increase d . The provisional child keeps the masses and halves the width, decreasing d by one. Every child has strictly smaller nonnegative rank R .

Induction on R now proves termination. Base cases require only finite arithmetic and, in the width case, the finite iteration of Lemma 2.1. At a nonbase node, every child terminates by induction. Lemma 4.2 then bounds the two passes; the remaining two children and local operations are finite as well. This reasoning makes no correctness assumption about a label. $\square$

The distinction between width reduction and budget reduction is shown in Figure 1. It permits induction for correctness and a sharper count than a general branching bound: only one child can preserve the mass budget, irrespective of the width.

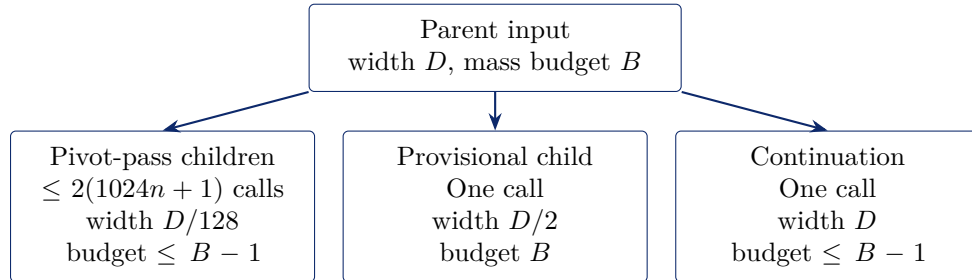


Figure 1: Children of a nonbase call. The two passes together produce at most $2(1024n + 1)$ narrow children, each decreasing B . The provisional child preserves B and halves the width; the continuation preserves the box and decreases B . Edges show the recursion tree, not the execution order.

5 Why the labels satisfy every witness

Termination supplies a well-founded order for the correctness proof. At a nonbase call, assume that all children satisfy the simultaneous labelling guarantee. We will show first that the marks I_- and I_+ can safely receive their prescribed labels. We will then show that every witness with a remaining claim survives the mass change. These two statements allow the continuation to finish the call.

In the next three lemmas, fix a nonbase input (A, D, a, b) and assume Theorem 2.4 for all its children. Witnesses, supports, and claims without a prime refer to this parent input. Peaks and gaps are those of Equations (3.2)–(3.3).

5.1 Protection during motion and mass at stopping

Lemma 5.1 (The downward pass). *For every relevant Max witness x , its peak $s_x(k)$ is constant throughout Step 3 and at most $D/2$. In particular, no Max witness claims an index in I_- . For every Min witness y that has a claim outside I_- ,*

$$b(\{i : g_i^y(k^\downarrow) < q\}) > 3/4. \quad (5.1)$$

Proof. Fix x and consider one downward update from pivot k . Its width- q translation is a witness for the child, since that child keeps Max masses a . By Lemma 3.1, every coordinate of gap at most $q/32 = p$ is a Max claim of the translated witness. The child's guarantee labels these coordinates $+$, and they do not move. In particular at least one gap-zero coordinate continues to attain the old peak.

At every other coordinate, the downward movement δ_i lies in $[0, p]$. For its gap $g_i^x(k) > p$,

$$x_i - k_i^{\text{new}} = s_x(k) - g_i^x(k) + \delta_i < s_x(k).$$

Thus no coordinate can overtake the old peak, even though many may move in the same update. The peak is unchanged. Initially $k = c$ and $x \leq A + D$, so $s_x(c) \leq D/2$. If x claims i ,

$$k_i^\downarrow \geq x_i - D/2 \geq A_i + D/2 - P = c_i - P,$$

which excludes i from I_- .

For the mass assertion, let y claim $j \notin I_-$. The stopped pivot satisfies $k_j^\downarrow \geq c_j - P$, and the claim gives $y_j \leq A_j + P$. Hence

$$s_y(k^\downarrow) \geq D/2 - 2P > D/2 - M. \quad (5.2)$$

A coordinate at the lower cap would have $k_i^\downarrow - y_i \leq D/2 - M$, since $y_i \geq A_i$. Every maximizer of $k_i^\downarrow - y_i$ is therefore strictly above its cap.

Suppose the mass in Equation (5.1) were at most $3/4$. The width- q Min translation has exactly that support, so it would have support mass at most one for the last child's masses $\frac{4}{3}b$. Its gap-zero maximizers would all have to receive $-$. At least one of those coordinates would then move down by a positive amount, because it is above the cap and $p > 0$. This contradicts the final unchanged update. Equality at $3/4$ also leads to this contradiction, proving the strict mass bound. $\square$

The downward marks are now safe, but the pivot will still move upward. The next lemma both protects the other type of claim and shows why the first mass certificate survives that later movement.

Lemma 5.2 (The upward pass). *For every relevant Min witness y , its peak $s_y(k)$ is constant throughout Step 4 and at most $D/2$. Thus no Min witness claims an index in I_+ . At the final pivot,*

$$b(\{i : g_i^y(k^f) < q\}) > 3/4 \quad \text{if } y \text{ has a claim outside } I_-; \quad (5.3)$$

$$a(\{i : g_i^x(k^f) < q\}) > 3/4 \quad \text{if } x \text{ has a claim outside } I_+. \quad (5.4)$$

Proof. In every upward child the Min masses are unchanged. Translate a fixed relevant Min witness y to that width- q box. All gaps at most p become Min claims, so those coordinates are labelled $-$ and stay fixed. Every other coordinate has gap greater than p and moves upward by at

most p . Its new deviation $k_i^{\text{new}} - y_i$ is therefore strictly below the old peak. A gap-zero coordinate does not move, proving exact peak preservation. At the start, $k^\downarrow \leq c$ and $y \geq A$ imply $s_y \leq D/2$. A Min claim at i consequently gives

$$k_i^f \leq y_i + D/2 \leq c_i + P,$$

so it is outside I_+ .

Each coordinate k_i is nondecreasing during this pass while s_y is fixed. It follows that

$$g_i^y(k^f) \leq g_i^y(k^\downarrow). \quad (5.5)$$

The set of Min gaps $< q$ can only grow. Its b -mass was greater than $3/4$ at $k^\downarrow$ whenever y had a claim outside I_- , by Lemma 5.1. Positivity of b proves Equation (5.3).

It remains to obtain the Max mass certificate at this second stopping point. If x claims $j \notin I_+$, then $x_j \geq A_j + D - P$ and $k_j^f \leq c_j + P$, giving

$$s_x(k^f) \geq D/2 - 2P > D/2 - M.$$

At an upper-capped coordinate, the inequalities $k_i^f = c_i + M$ and $x_i \leq A_i + D$ would give a deviation at most $D/2 - M$. Thus every maximizer lies strictly below the upper cap. If the a -mass of the gaps $< q$ were at most $3/4$, the width- q translation would be a Max witness for the final child's masses $\frac{4}{3}a$. Its maximizers would receive $+$ and force a positive upward movement, contradicting the unchanged update. This proves Equation (5.4). $\square$

The mass bounds are absolute: each qualifying set has mass greater than $3/4$, while the entire witness support has mass at most one. They are not merely fractional lower bounds relative to a support that might have much smaller mass. This strength is exactly what the reweighting calculation needs.

5.2 Preserving the unsettled witnesses

The provisional child does not directly label the parent's claims. Its purpose is to assign the favorable label on each large-mass set identified above. The larger width $D/2$ makes these sets claims of the translated witnesses.

Lemma 5.3 (Witness survival). *Every parent Max witness with a claim outside I_+ remains a Max witness for $(A, D, \bar{a}, \bar{b})$. Every parent Min witness with a claim outside I_- remains a Min witness for that input.*

Proof. Write $r = D/2$ for the provisional width. Since

$$q = D/128 < r/32 = D/64,$$

the translated width- r witness claims every coordinate whose parent gap is $< q$.

For a Max witness x as in the statement, let $N_x = \{i : g_i^x(k^f) < q\}$ and $Q_+ = \{i : \lambda_i^0 = +\}$. Lemma 5.2 gives $a(N_x) > 3/4$. By Lemma 3.1, $N_x \subseteq S_x$; its coordinates are claims of the width- r translation, which qualifies for the unchanged masses a . Correctness of that child therefore implies $N_x \subseteq Q_+$. Reweighting the original support gives

$$\begin{aligned} \bar{a}(S_x) &= \frac{8}{5}a(S_x) - \frac{4}{5}a(S_x \cap Q_+) \\ &< \frac{8}{5} \cdot 1 - \frac{4}{5} \cdot \frac{3}{4} = 1. \end{aligned}$$

The continuation has the same box and operator, so the subsolution inequalities are unchanged. The displayed support bound is all that is needed to keep x a witness.

For Min set $N_y = \{i : g_i^y(k^f) < q\}$ and $Q_- = \{i : \lambda_i^0 = -\}$. The same two lemmas show $N_y \subseteq S_y \cap Q_-$ and $b(N_y) > 3/4$. Hence

$$\bar{b}(S_y) = \frac{8}{5}b(S_y) - \frac{4}{5}b(S_y \cap Q_-) < \frac{8}{5} - \frac{4}{5} \cdot \frac{3}{4} = 1.$$

The supersolution inequalities still hold in the unchanged box, so y also survives. $\square$

5.3 Completing the simultaneous induction

Proof of Theorem 2.4. Use induction on $R = d + B(\mu)$ from Lemma 4.3, which has already established termination. We prove that a single returned vector meets every parent obligation.

In the mass base case, $a_i b_i > 1$ at every vertex. If $b_i > 1$, no Min witness can contain i in its support, and hence no Min witness can claim it; returning $+$ is permitted. If $b_i \leq 1$, then $a_i > 1$, so no Max witness can claim that vertex; returning $-$ is permitted. This proves all simultaneous obligations in Step 1.

In the width base case, comparison with the computed box solution z gives $x \leq z \leq y$ for every Max witness x and Min witness y . At a Max claim,

$$z_i \geq A_i + D - P > A_i + D/2,$$

and Step 2 returns $+$. At a Min claim, $z_i \leq A_i + P < A_i + D/2$, and it returns $-$. The argument also applies to $D = 1$, for which the midpoint is a rational threshold rather than an integer coordinate.

In a nonbase call, all children have smaller rank and satisfy the simultaneous guarantee by induction. The hypotheses of Lemmas 5.1–5.3 therefore hold. Fix any Max witness and any index i it claims. Lemma 5.1 excludes i from I_- . If $i \in I_+$, the positive override settles it. Otherwise that witness has a claim outside I_+ and survives reweighting by Lemma 5.3. The continuation must label i positively, and no override changes it.

For any Min claim, Lemma 5.2 excludes I_+ . A claim in I_- receives the negative override. A claim outside I_- belongs to a surviving Min witness and receives $-$ from the continuation, with no later change. The marks I_- refer to the earlier pivot, but their safety was proved for the fixed parent witnesses and remains valid after the upward pass. The two marked sets may intersect; that intersection contains no claim of either type, so the stipulated negative precedence causes no conflict.

Every vertex receives exactly one label. Since the witnesses and claimed indices in the preceding argument were arbitrary, all their obligations hold together. This completes the induction. $\square$

6 Counting calls and bit operations

Correctness now gives the complete winning set. To finish the theorem, we must bound the work in terms of binary input length rather than the numerical width of the box. There are two relevant depths: the width can be halved polynomially many times in L , but the mass budget can decrease only logarithmically many times in n . Because only one child preserves that budget, these depths give a quasipolynomial call count. We then bound the exact arithmetic within each call.

6.1 The recursion tree

For the initial data of Equation (2.9), write

$$d_0 = \log_2 D_0, \quad B_0 = B(1/n^2), \quad H = d_0 + B_0, \quad s = 2048n + 3. \quad (6.1)$$

The minimum initial mass product is $1/n^2$. Every path in the call tree has at most H edges by the rank decrease of Lemma 4.3. At most B_0 of its edges can decrease B , even if an individual edge decreases it by more than one. At any node there is at most one child preserving B , namely the provisional child. There are at most $2(1024n + 1)$ loop children and one continuation, so at most s children decrease B .

Lemma 6.1 (Number of calls). *The complete main computation makes at most*

$$\sum_{\ell=0}^H \sum_{j=0}^{\min(\ell, B_0)} \binom{\ell}{j} s^j \leq (H+1)(B_0+1)(1+Hs)^{B_0} \quad (6.2)$$

calls, including the initial call.

Proof. Consider a node at depth ℓ whose path uses j budget-decreasing edges. Their positions can be chosen in $\binom{\ell}{j}$ ways. Order each node's budget-decreasing children by execution order. At each chosen position there are at most s choices of that ordinal; at each other position there is at most one. This specifies at most $\binom{\ell}{j} s^j$ possible nodes. Sum over all depths $0 \leq \ell \leq H$ and $0 \leq j \leq \min(\ell, B_0)$. For the second inequality, each summand is at most $(Hs)^j \leq (1+Hs)^{B_0}$ and there are at most $(H+1)(B_0+1)$ summands. $\square$

This count uses the same recursion principle as the reduced-precision analysis of Parys [25, Section 5], further developed in [20]: one child retains the bounded parameter while all others improve it. Here the two parameters are the logarithm of an integer width and the budget derived from rational mass products.

The explicit encoding bounds both n and $m = |E|$ polynomially in $L + 1$, and each original weight has $O(L + 1)$ bits. The perturbation and Equation (2.5) imply

$$\log_2(W + 1) = O(L + \log(n + 1)), \quad d_0 = O(L + \log(n + 1)).$$

Since $\rho = 32/25$ is a fixed constant greater than one,

$$B_0 = 1 + \lfloor \log_\rho(n^2) \rfloor = O(1 + \log(n + 1)).$$

Thus H and s are polynomial in $L + 1$, and $B_0 = O(\log(L + 2))$. Taking logarithms in the call bound gives

$$\begin{aligned} \log_2(\text{number of calls}) &\leq \log_2(H + 1) + \log_2(B_0 + 1) + B_0 \log_2(1 + Hs) \\ &= O((\log_2(L + 2))^2). \end{aligned} \quad (6.3)$$

This includes $n = 1$, for which $B_0 = 1$.

6.2 Exact representations

The nested-box property keeps every endpoint and pivot of the main computation inside $[0, D_0]^V$. Each such coordinate and each width therefore uses $O(d_0 + 1)$ bits. In a basic iteration, an intermediate edge expression is a coordinate plus a transformed weight. It may be negative, but its signed binary length is $O(d_0 + \log(W + 1) + 1)$, polynomial in $L + 1$.

Represent a mass by a positive numerator and a positive denominator; reducing fractions is unnecessary. Starting from $1/n$, a mass is multiplied along any path only by numbers in

$$\{1, 4/3, 4/5, 8/5\}.$$

Only budget-decreasing edges change the masses, and a path contains at most B_0 such edges. Thus each numerator and denominator has $O(B_0 + \log(n + 1))$ bits. Children receive their own copies of the input masses. In particular, completing one sibling does not multiply the parent masses before another sibling starts. Testing $a_i b_i > 1$ uses integer cross-multiplication, and testing $b_i > 1$ or applying a constant multiplier also uses numbers of polynomial length. All mass arithmetic is exact.

The proof quantities B and μ , and the peaks, gaps, supports, and witness vectors, need not be stored or computed. The algorithm uses neither a logarithm evaluation for its rank nor an oracle for the existence of a witness. The procedure evaluates F only inside **Basic**, by explicit max/min scans of the edge list.

6.3 Local cost and uniform implementation

Each call has polynomial nonrecursive work. The mass and width tests are finite scans with the arithmetic just bounded. If the width base case applies, $D < 4096$, and the imported iteration bound gives

$$1 + nD \leq 1 + 4095n$$

evaluations, including the final stability test. Every evaluation scans the m explicit edges, adds their transformed weights, takes the relevant extrema, and clips at n endpoints. It acts on integers of polynomial length. The midpoint comparison is implemented as $2z_i > 2A_i + D$, which also handles $D = 1$.

In a nonbase call, the constants defining the pivot are obtained by exact divisions by powers of two. Each pass has $O(n + 1)$ iterations apart from its children's work. A scan of the n coordinates performs one simultaneous update and tests whether it changed anything. Preparing child inputs and receiving their labels, forming I_- and I_+ , reweighting, and applying the final overrides likewise use polynomially many operations on polynomial-length integers. Schoolbook arithmetic suffices throughout.

A depth-first implementation runs children sequentially and retains one record per active call. The record holds the parent masses, endpoints, pivot, labels, and current loop position. There are at most $H + 1$ active records, each of polynomial size. Stack management is therefore included in the same polynomial local bound. Preprocessing the transformed weights, finding the least power of two in Equation (2.5), and creating masses $1/n$ take polynomial time. Reading the input and producing the n final labels also do so.

Multiply this polynomial per-call cost by the call count bounded in Equation (6.3), and include preprocessing and output. For an absolute constant $C > 0$, the complete bound is

$$2^{C(\log_2(L+2))^2}. \tag{6.4}$$

The constants in the procedure are fixed independently of the game, and every arithmetic and control operation has been specified. This gives a uniform deterministic bit algorithm.

Proof of Theorem 1.1. Compute Equation (2.9), run **Label**, and output the positively labelled vertices. Theorem 2.4 guarantees the labels required by every witness. In the initial box, the box solution is a witness of both types, and the argument after Equation (2.9) identifies all its required labels with the bands of Proposition 2.2. The positive band is exactly the zero-threshold winning set, including against every history-dependent opponent. Equation (6.4) bounds the entire computation. Signed binary weights, self-loops, and separately listed parallel edges are all covered by the construction and accounting above. $\square$

7 Exact values and optimal positional strategies

We now use the winning-set algorithm to recover the numerical values and strategies of the game. For the input model of Theorem 1.1, write

$$\text{val}_w(v) = \sup_{\sigma} \inf_{\tau} \text{MP}_w(\pi_{v,\sigma,\tau}),$$

where the strategies may depend on the full finite history.

Corollary 7.1 (Full game solution). *A uniform deterministic algorithm returns the exact value $\text{val}_w(v)$ as a reduced rational for every $v \in V$ and one positional strategy σ^* for Max and one positional strategy τ^* for Min such that, simultaneously for every $v \in V$ and every pair of history-dependent strategies σ, τ ,*

$$\text{MP}_w(\pi_{v,\sigma^*,\tau}) \geq \text{val}_w(v), \quad \text{MP}_w(\pi_{v,\sigma,\tau^*}) \leq \text{val}_w(v).$$

The complete computation takes $2^{O((\log_2(L+2))^2)}$ bit operations.

Proof. Positional values. The uniform form of Ehrenfeucht–Mycielski positional determinacy [13], stated explicitly in [7, Theorem 1 and the following paragraph], provides one positional strategy for each player attaining the value from all vertices at once, against arbitrary history-dependent opponents. The latter source uses an edge relation without parallel entries. For our convention, replace each explicit edge $e : u \rightarrow v$ by $u \rightarrow x_e \rightarrow v$ through a fresh vertex with a single outgoing edge, with weights $2w(e)$ and 0. Keep ownership at the original vertices; the owner of x_e is immaterial. The resulting graph has no parallel edges or loops. Expanded histories retain the identity of every original edge. Their averages at even prefixes equal the original averages, and the odd-prefix discrepancy tends to zero. Thus payoffs and the uniform positional guarantees transfer back to our graph, including loops and parallel entries.

Set $W_{\text{in}} = \max_{e \in E} |w(e)|$. Fix both uniform optimal positional strategies. From any vertex the resulting play eventually repeats a simple directed cycle of at most n edges in the original graph, and its mean is the value at that vertex. Consequently every value has a reduced representation a/b with

$$1 \leq b \leq n, \quad |a| \leq bW_{\text{in}}, \quad -W_{\text{in}} \leq a/b \leq W_{\text{in}}.$$

The same bounds hold after restricting outgoing choices, provided at least one outgoing edge remains at every vertex.

Threshold queries and exact reconstruction. We use the classical reduction from values to threshold queries [7, Section 5.2], keeping track of binary lengths. For integers p and $q > 0$, replace each weight by $w_{p/q}(e) = qw(e) - p$. For every infinite play,

$$\text{MP}_{w_{p/q}}(\pi) = q \text{MP}_w(\pi) - p.$$

Positional attainment therefore makes the winning set returned by Theorem 1.1 exactly $\{v : \text{val}_w(v) \geq p/q\}$.

Choose $K = 1 + \lceil \log_2(2(W_{\text{in}} + 1)n^2) \rceil$, computable from integer bit lengths. For each vertex, start with the closed interval $[-W_{\text{in}}, W_{\text{in}}]$ and make K bisections. Query its midpoint and move the lower endpoint to the midpoint if the vertex belongs to the returned set; otherwise move the upper endpoint. The final interval contains its value and has width $2W_{\text{in}}2^{-K} < 1/n^2$. Distinct reduced rationals with positive denominators at most n differ by at least $1/n^2$. To reconstruct the value from an interval $[\ell, h]$, for each $1 \leq b \leq n$ test the integer $a = \lceil b\ell \rceil$ for $a/b \leq h$, reduce the retained fractions, and remove repetitions. Since $b(h - \ell) < 1$, this tests every possible numerator. The value bound above guarantees exactly one resulting rational. Call this value-vector procedure $\text{Values}(H)$ for any restricted game H ; the original n, W_{in}, K remain valid for all such calls.

Separate strategy self-reductions. An edge joining vertices of the same value need not be optimal. For example, let a Max vertex u have a zero-weight self-loop and a zero-weight edge to a vertex v whose only edge is a self-loop of weight 1. Both vertices have value 1, but staying at u yields payoff 0. We therefore test restrictions of the current game and preserve its entire value vector.

First compute $\mathbf{v} = \text{Values}(G)$. For Max, start from a fresh copy $H = G$ and process the Max vertices in any fixed order. At a vertex u , form a trial H_e for each outgoing edge e by keeping only e at u . Keep a trial for which $\text{Values}(H_e) = \mathbf{v}$, and continue from that trial. Such an edge always exists. Indeed, take the edge chosen at u by a globally optimal positional Max strategy of the current H . Restricting Max's choices cannot increase any value, while that strategy remains available and secures the current entire value vector, so no value decreases either. This proves the invariant $\text{Values}(H) = \mathbf{v}$.

At the end, Max has one allowed edge at every owned vertex, whereas all of Min's original choices remain. The resulting single Max strategy therefore secures $\mathbf{v}$ from every vertex against every Min strategy. To recover Min's strategy, start again from a fresh copy of the original G , not from the Max-restricted game. At each Min vertex keep an outgoing edge whose trial preserves $\mathbf{v}$. A globally optimal positional Min strategy of the current game supplies such an edge: restricting Min cannot decrease values, and that strategy prevents any increase. The final Min strategy is optimal against every original Max strategy. Thus the two separately recovered strategies have the simultaneous guarantees in the statement.

Call and bit bounds. Put $m = |E|$. At a vertex, at most its original outdegree many trials are tested, so the two self-reductions together use at most m trial value-vector computations. Including the original computation, there are at most

$$Q = nK(1 + m) \tag{7.1}$$

threshold calls. Every midpoint has denominator dividing 2^K and a numerator of $O(\log(W_{\text{in}}+1)+K)$ bits. Each transformed weight has the same order of bit length. Since $K = O(L + \log(n+1))$, the full encoding length of every queried game is $O(L + mK)$, polynomial in $L + 1$. Reconstruction, reduced-rational comparison, and copying restricted edge lists also have polynomial bit cost. The

explicit encoding bounds n, m polynomially in $L + 1$, so Q is polynomial. Applying Theorem 1.1 to these polynomial-length games and multiplying by Q gives $2^{O((\log_2(L+2))^2)}$ bit operations, including the polynomial-size value and strategy output. $\square$

The preceding reduction also applies to the companion algorithm. Here the additional issue is that threshold queries depend on earlier answers, so the winning-set error must be controlled at each query.

Corollary 7.2 (Randomized full game solution). *For the game model of Theorem 1.1, a uniform randomized algorithm returns all exact values and both globally optimal positional strategies specified in Corollary 7.1 with probability at least $7/8$ for the entire output. Its running time is $2^{O((\log_2(L+2))^2)}$ bit operations on every random tape.*

Proof. The companion's Theorem 1.1 [24] returns the whole winning set correctly with probability at least $7/8$ and has the quasipolynomial bit bound on every tape. Use the reduction above, whose number of threshold queries is bounded by the polynomial Q in Equation (7.1). For each query, take $r = 2\lceil \log_2(8Q) \rceil + 1$ independent fresh runs and use the coordinatewise majority of their winning sets. Conditional on the previous history, the current query is fixed. The amplified set is correct whenever more than half of the runs return the whole correct set, and its conditional failure probability is at most

$$\sum_{j=\lceil r/2 \rceil}^r \binom{r}{j} (1/8)^j \leq 2^r (1/8)^{r/2} = 2^{-r/2} < \frac{1}{8Q}.$$

A first-error union bound over the adaptive queries makes every threshold answer correct with probability at least $7/8$. On this event the preceding deterministic argument gives all exact values and both globally optimal strategies. On other tapes, if reconstruction does not yield a unique rational or no value-preserving trial is found, the reduction stops with zero at every value coordinate and the first listed outgoing edge at every owned vertex. All loops have the stated caps even then. Since $r = O(\log(L + 2))$ and all queried encodings retain their polynomial bound on every tape, the total bit bound remains quasipolynomial on every tape, including random-bit generation. This transfer uses only the winning-set interface. $\square$

8 Fixed- and minimum-credit energy games

The preceding algorithms also compute the initial credits needed in one-resource energy games. Retain the game model of Theorem 1.1. For a start $v \in V$ and an integer credit $c \geq 0$, call (v, c) *energy-winning* if Max has a strategy such that, against every history-dependent Min strategy, the induced play satisfies

$$c + \sum_{t=0}^{j-1} w(e_t) \geq 0 \quad \text{for every } j \geq 0. \quad (8.1)$$

There is one accumulated resource and no upper bound or saturation. Write $c_{\min}(v)$ for the least nonnegative integer credit for which (v, c) is energy-winning, and put $c_{\min}(v) = \infty$ if there is none. A positional Max policy specifies one labelled outgoing edge at each Max vertex.

The reductions connecting these problems to mean-payoff games are classical. Bouyer, Fahrenberg, Larsen, Markey, and Srba formulate the fixed-credit lower-bound problem and reduce it to mean payoff using adversary-controlled returns to the start [6, Section 2.1 and Proposition 12].

Our construction places the initial credit on each return edge. Brim, Chaloupka, Doyen, Gentilini, and Raskin distinguish the unknown-credit and minimum-credit problems, record the unknown-credit/mean-payoff equivalence, and characterize minimum credits by an energy progress measure [7, Section 2 and Theorems 3 and 5]. The reset construction below lets us apply Corollary 7.1 to a prescribed-credit instance, recovering a policy when that instance is winning. A bound on finite minimum credits then permits exact binary search.

Corollary 8.1 (Fixed and minimum energy credits). *Uniform deterministic algorithms have the following guarantees.*

1. *Given v and a nonnegative binary integer c , decide whether (v, c) is energy-winning. On a positive answer, return a positional Max policy satisfying Equation (8.1) from that start and credit against every history-dependent Min strategy.*
2. *Given v , return the exact $c_{\min}(v)$, including the possible answer ∞ . On a finite answer, return a positional Max policy winning from v at credit $c_{\min}(v)$.*

The fixed-credit computation takes $2^{O((\log_2(L_c+2))^2)}$ bit operations, where L_c is the full explicit input length including the start and credit. The minimum-credit computation takes $2^{O((\log_2(L+2))^2)}$ bit operations for the original game length L . Handling every vertex by separate per-start searches, including a separate policy for every finite answer, has the same form of total bit bound.

Each policy may depend on the queried start and credit; it also wins from that same start at every larger credit.

Proof. Fixed credit by resetting. For the fixed pair (v, c) construct a mean-payoff game $R_{v,c}$. Replace every $u \in V$ by an entry u_{in} owned by Min and an exit u_{out} owned by the original owner of u . Add the two edges

$$u_{\text{in}} \xrightarrow{c} v_{\text{in}}, \quad u_{\text{in}} \xrightarrow{0} u_{\text{out}},$$

and replace each labelled original edge $e : u \rightarrow z$ by its own edge

$$u_{\text{out}} \xrightarrow{w(e)} z_{\text{in}}.$$

Start at v_{in} . The game is total and has $2n$ vertices and $m + 2n$ edges, where $m = |E|$. The reset at $u = v$ is a nonnegative self-loop, including when $c = 0$, and distinct original parallel edges remain distinct. If $U(R_{v,c})$ denotes its zero-threshold winning set, then

$$(v, c) \text{ is energy-winning in } G \iff v_{\text{in}} \in U(R_{v,c}). \quad (8.2)$$

For the forward implication, use an energy-winning strategy from (v, c) and restart its history after each reset. In any transformed prefix, every completed segment has original-edge sum at least $-c$ and then receives the reset weight $+c$. Its total is therefore nonnegative. The unfinished segment has sum at least $-c$; the intervening zero edges do not change this bound. Every transformed prefix sum is consequently at least $-c$. Dividing by its length shows that the lower limiting average is nonnegative. This argument also covers no resets and infinitely many resets, and does not assume that the original energy strategy is positional.

Conversely, if $v_{\text{in}} \in U(R_{v,c})$, Corollary 7.1 returns a positional Max strategy for $R_{v,c}$ guaranteeing nonnegative mean payoff from that start. Restrict its choices at Max exits to obtain a positional policy in G . Suppose a consistent original prefix of length j from v had weight $q < -c$. In the reset game Min can choose continuation at its entries, follow that prefix at its owned exits, and then reset. This is a closed walk from v_{in} with weight $q + c < 0$ and length $2j + 1$. Positionality

of the Max strategy makes the same walk consistent on every repetition. Min could therefore repeat it forever, giving mean payoff $(q + c)/(2j + 1) < 0$, a contradiction. Thus all original prefixes have weight at least $-c$. This proves the equivalence and the positive-answer policy guarantee; only the Max strategy from the full-solution call is used.

A finite credit bound and exact search. For the original weights put

$$W_{\text{in}} = \max_{e \in E} |w(e)|, \quad H = (n - 1)W_{\text{in}}. \quad (8.3)$$

In particular W_{in} and H may be zero. An energy winner with any finite credit is an original zero-threshold mean-payoff winner, since its prefix sums are bounded below. Conversely, the cycle-deletion estimate in the proof of Proposition 2.2 gives a positional Max policy on the original winning region for which every consistent prefix has weight at least $-H$. Indeed, its permitted simple cycles have nonnegative original sum, and deleting such cycles from a prefix leaves at most $n - 1$ edges, each of weight at least $-W_{\text{in}}$. Consequently, for the original winning set $U(G)$,

$$c_{\min}(v) < \infty \iff v \in U(G), \quad v \in U(G) \implies 0 \leq c_{\min}(v) \leq H. \quad (8.4)$$

The algorithm uses this existence bound as a known winning endpoint; it need not compute the potential used in that proposition.

Compute $U(G)$ once by Theorem 1.1 and output ∞ for starts outside it. For a winning start, the fixed-credit predicate is monotone: a strategy that wins at c still wins at every larger credit. Binary search with a formal losing lower endpoint $\ell = -1$ and the known winning upper endpoint $h = H$. While $h - \ell > 1$, query $\lfloor (\ell + h)/2 \rfloor$ by Equation (8.2) and replace the endpoint having the same answer. The sentinel itself is never queried. This takes at most $\lceil \log_2(H + 1) \rceil$ fixed-credit decisions and returns the exact least winning integer. When $H = 0$ the initial gap is one, so no search query is made and the answer is zero. This includes both $n = 1$ and all-zero weights; any start without a finite credit has already been excluded by $U(G)$. To return a policy at the minimum, apply Corollary 7.1 to $R_{v, c_{\min}(v)}$ and restrict its Max choices as above.

Bit complexity and output size. Let b_c be the binary length of the supplied nonnegative credit, so $b_c = O(1 + \log_2(c + 1))$. The explicit reset game has encoding length

$$O(L + n(b_c + \log_2(n + 1))),$$

which is polynomial in L_c . The credit is copied into only n edge records; no array indexed by its numerical value is constructed. Theorem 1.1 decides membership in its winning set, and a single application of Corollary 7.1 on a positive answer supplies the policy, within the fixed-credit bound claimed.

For minimum credits, H and every queried credit have $O(1 + \log_2(n + 1) + \log_2(W_{\text{in}} + 1)) = O(L + 1)$ bits. All queried encodings therefore have polynomial length in $L + 1$. For all vertices, the outer reduction makes at most $1 + n \lceil \log_2(H + 1) \rceil$ direct winning-set calls, including the original one, and at most one full-solution call for each requested finite-credit policy. These counts are polynomial in $L + 1$. Polynomially many calls on polynomial-length inputs preserve $2^{O((\log_2(L+2))^2)}$ bit complexity. Computing H , the binary search arithmetic, and copying the policies have polynomial bit cost. The output also has polynomial length: each finite credit has the stated bit bound, and each per-start policy lists one original edge at each Max vertex. No energy-indexed array is used in the search. $\square$

9 Tropical feasibility and fixed corank

Akian, Gaubert, and Guterman identify the coordinates that can be finite in a solution of tropical inequalities with the winning states of an associated mean-payoff game [1, Theorem 3.2 and Corollary 3.7]. Together with Theorem 1.1, this gives binary-length bounds for tropical feasibility and for rank within a fixed distance of the number of columns. We use the conventions $a + (-\infty) = -\infty$ and $\max \emptyset = -\infty$.

Corollary 9.1 (Tropical feasibility and fixed corank). *For explicitly listed $m \times n$ matrices A, B and m -dimensional vectors c, d over $\mathbb{Q} \cup \{-\infty\}$, feasibility of*

$$\max\{c_i, \max_j(A_{ij} + x_j)\} \leq \max\{d_i, \max_j(B_{ij} + x_j)\} \quad \text{for every row } i$$

is decidable deterministically in $2^{O((\log(L+2))^2)}$ bit operations, where L is the complete explicit binary input length. Variables range over $\mathbb{R} \cup \{-\infty\}$; feasibility with all coordinates finite is decidable within the same bound. Equalities between the two displayed maxima are allowed, interpreted as pairs of inequalities. For a homogeneous system ($c = d = -\infty$), one can also compute the maximal finite-coordinate support of a solution, including the empty support. In particular, the same bound decides feasibility of max-atom systems $z \leq \max(x, y) + a$, with rational constants a and finite real, equivalently rational, variables.

For each fixed integer $k \geq 0$, a deterministic algorithm with bit bound $2^{O_k((\log(L+2))^2)}$ decides whether an explicitly listed $m \times n$ matrix over $\mathbb{Q} \cup \{-\infty\}$, with $m \geq n \geq 1$, has tropical rank at least $n - k$. Here tropical rank is the largest order of a square submatrix whose permutation sums have a finite, unique maximum. The rank is zero if no such nonempty submatrix exists.

Proof. Integer data and homogeneous supports. Let Q be the product of the positive denominators of the finite input coefficients, taking $Q = 1$ when there are none. Multiplying every finite coefficient and variable by Q preserves the inequalities and the sets of finite coordinates. It also preserves the comparisons between permutation sums that define tropical rank. The bit length of Q is at most one plus the sum of the denominator bit lengths, so the scaled integer data have polynomial binary length.

Consider first a homogeneous system

$$\max_j(C_{ij} + z_j) \leq \max_j(D_{ij} + z_j) \quad \text{for every } i. \quad (9.1)$$

The support of z is $\{j : z_j \neq -\infty\}$. To obtain a total game, apply the preprocessing of [1, Section 2.1]. Discard a row whose left side is identically $-\infty$. If a right side is identically $-\infty$, every variable with a finite coefficient on that row's left side must equal $-\infty$. Record these coordinates, substitute $-\infty$ for them throughout the system, and repeat. Each step removes a row or a variable, so this takes polynomial time. If no variables remain, the maximal support is empty. Otherwise add the tautologies $z_j \leq z_j$ for the remaining variables.

The resulting bipartite game has a Min vertex for each remaining column and a Max vertex for each row. A finite C_{ij} gives an edge from column j to row i of weight $-C_{ij}$; a finite D_{ij} gives an edge from row i to column j of weight D_{ij} . Every vertex has an outgoing edge. Theorem 3.2 of [1] identifies its winning column vertices with the maximal support of a solution. The recorded deleted coordinates belong to no support. The union of all supports is attained, because coordinatewise maxima of finitely many solutions are again solutions; the identically $-\infty$ solution realizes the empty case.

The cited game counts a column–row–column pair as one turn. Counting the two edges separately halves the mean payoff; the contribution of an unmatched edge tends to zero. The source uses upper limiting averages; positional optimality makes their nonnegative winning set coincide with that for our lower limiting averages. The game has polynomially many vertices and edges, with weights copied or negated from the scaled coefficients or equal to zero. Its full binary encoding is polynomial in $L + 1$, so Theorem 1.1 computes the maximal support within the asserted bound.

Affine and finite feasibility. After scaling, append c to A and d to B as a new column, and introduce one coordinate t . The homogeneous system is

$$\max\{c_i + t, \max_j(A_{ij} + z_j)\} \leq \max\{d_i + t, \max_j(B_{ij} + z_j)\}.$$

Every affine solution x gives a homogeneous solution $(x, 0)$. Conversely, a homogeneous solution with finite t gives the affine solution $x_j = z_j - t$. Thus affine feasibility asks whether the new coordinate belongs to the maximal support. Feasibility with all original coordinates finite asks whether that support contains all $n + 1$ coordinates. A coordinate eliminated during preprocessing is absent from the computed support, so any test requiring it fails; this includes t if it is forced to $-\infty$. This is the homogenization underlying [1, Theorem 3.5 and Corollaries 3.4 and 3.7]. Equalities are handled by imposing both inequalities.

A max-atom $z \leq \max(x, y) + a$ is a homogeneous row with one term on the left and two on the right. Apply the all-finite test. With integer coefficients, taking the floor of each finite coordinate preserves every inequality: flooring commutes with integer translation and with finite maxima. Scaling back therefore gives a rational solution whenever a real one exists. This also explains the integral-solution interface in [1, Proposition 3.9, Corollary 3.10 and Remark 3.15].

Fixed corank. If $n \leq k$, the rank threshold is nonpositive and the answer is yes. Otherwise set $r = n - k$. The rank criterion of [1, Corollaries 4.16–4.17] says that the rank is at least r exactly when some set of r columns is tropically independent. For each such set let M be the resulting $m \times r$ matrix. Its columns are dependent precisely when there is a vector $z \not\equiv -\infty$ such that every row maximum is attained at least twice or equals $-\infty$. Equivalently, z satisfies the homogeneous system

$$M_{ij} + z_j \leq \max_{\ell \neq j}(M_{i\ell} + z_\ell) \quad (1 \leq i \leq m, 1 \leq j \leq r). \quad (9.2)$$

Its maximal support is therefore empty exactly when the selected columns are independent. This is the inequality form of the game reduction in [1, Theorem 4.9]. The row index (i, j) retains the column excluded from Max’s next choice. Empty right sides and all- $-\infty$ columns are covered by the support computation above, including when $r = 1$.

There are at most mr inequalities, with r coefficients on each side, so each test has polynomial encoding length. There are $\binom{n}{r} = \binom{n}{k} \leq n^k$ tests. For fixed k , their total bit cost is $2^{O_k((\log(L+2))^2)}$, as claimed. $\square$

Strongly polynomial algorithms have been claimed using max-atoms and tropical optimization [27, 28]. Appendix A of the randomized companion gives a two-variable counterexample to the printed variable-selection rule in arXiv:2603.26423v4 [24]. That comparison is specific to the version and rule examined; it neither assesses the distinct 2025 max-atom proof nor transfers to a revised procedure. It has no role in the proofs of this article.

References

- [1] Marianne Akian, Stéphane Gaubert, and Alexander Guterman. Tropical polyhedra are equivalent to mean payoff games. *International Journal of Algebra and Computation*, 22(1):1250001, 2012.
- [2] André Arnold, Damian Niwiński, and Paweł Parys. A quasi-polynomial black-box algorithm for fixed point evaluation. In *29th EACSL Annual Conference on Computer Science Logic (CSL 2021)*, volume 183 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 9:1–9:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021.
- [3] Peter Austin and Daniele Dell’Erba. Errata to: “Faster Deterministic Exponential Time Algorithm for Energy Games and Mean Payoff Games”. arXiv:2310.04130v1, October 6, 2023.
- [4] Henrik Björklund, Sven Sandberg, and Sergei Vorobyov. A combinatorial strongly subexponential strategy improvement algorithm for mean payoff games. In *Mathematical Foundations of Computer Science 2004*, volume 3153 of *Lecture Notes in Computer Science*, pages 673–685. Springer, 2004. DOI: [10.1007/978-3-540-28629-5_52](https://doi.org/10.1007/978-3-540-28629-5_52).
- [5] Henrik Björklund and Sergei Vorobyov. A combinatorial strongly subexponential strategy improvement algorithm for mean payoff games. *Discrete Applied Mathematics*, 155(2):210–229, 2007. DOI: [10.1016/j.dam.2006.04.029](https://doi.org/10.1016/j.dam.2006.04.029).
- [6] Patricia Bouyer, Uli Fahrenberg, Kim G. Larsen, Nicolas Markey, and Jiří Srba. Infinite runs in weighted timed automata with energy constraints. In Franck Cassez and Claude Jard, editors, *Formal Modeling and Analysis of Timed Systems (FORMATS 2008)*, volume 5215 of *Lecture Notes in Computer Science*, pages 33–47. Springer, 2008. DOI: [10.1007/978-3-540-85778-5_4](https://doi.org/10.1007/978-3-540-85778-5_4).
- [7] Luboš Brim, Jakub Chaloupka, Laurent Doyen, Raffaella Gentilini, and Jean-François Raskin. Faster algorithms for mean-payoff games. *Formal Methods in System Design*, 38(2):97–118, 2011. DOI: [10.1007/s10703-010-0105-x](https://doi.org/10.1007/s10703-010-0105-x).
- [8] Michaël Cadilhac, Antonio Casares, and Pierre Ohlmann. Fast value iteration: A uniform approach to efficient algorithms for energy games. In Arie Gurfinkel and Marijn Heule, editors, *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2025)*, Part II, volume 15697 of *Lecture Notes in Computer Science*, pages 323–342. Springer, 2025. DOI: [10.1007/978-3-031-90653-4_16](https://doi.org/10.1007/978-3-031-90653-4_16).
- [9] Cristian S. Calude, Sanjay Jain, Bakhadyr Khoussainov, Wei Li, and Frank Stephan. Deciding parity games in quasi-polynomial time. *SIAM Journal on Computing*, 51(2):STOC17–152–STOC17–188, 2022. DOI: [10.1137/17M1145288](https://doi.org/10.1137/17M1145288).
- [10] Laure Daviaud, Marcin Jurdziński, and Ranko Lazić. A pseudo-quasi-polynomial algorithm for mean-payoff parity games. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS ’18*, pages 325–334. ACM, 2018. DOI: [10.1145/3209108.3209162](https://doi.org/10.1145/3209108.3209162).
- [11] Dani Dorfman, Haim Kaplan, and Uri Zwick. A faster deterministic exponential time algorithm for energy games and mean payoff games. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, volume 132 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 114:1–114:14. Schloss Dagstuhl – Leibniz-Zentrum

für Informatik, 2019. An updated author version contains the weight-independent bound in Theorem 3.10: <https://danidorfman.com/publication/energy-games/energy-games.pdf>.

- [12] Dani Dorfman, Haim Kaplan, and Uri Zwick. Improved bounds for strategy improvement algorithms for energy games. In Philip Bille, Seth Pettie, and Sabine Storandt, editors, *34th Annual European Symposium on Algorithms (ESA 2026)*, volume 388 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 140:1–140:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026. DOI: [10.4230/LIPIcs.ESA.2026.140](https://doi.org/10.4230/LIPIcs.ESA.2026.140).
- [13] Andrzej Ehrenfeucht and Jan Mycielski. Positional strategies for mean payoff games. *International Journal of Game Theory*, 8(2):109–113, 1979. DOI: [10.1007/BF01768705](https://doi.org/10.1007/BF01768705).
- [14] Nathanaël Fijalkow, Paweł Gawrychowski, and Pierre Ohlmann. Value iteration using universal graphs and the complexity of mean payoff games. In *45th International Symposium on Mathematical Foundations of Computer Science (MFCS 2020)*, volume 170 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 34:1–34:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020. DOI: [10.4230/LIPIcs.MFCS.2020.34](https://doi.org/10.4230/LIPIcs.MFCS.2020.34).
- [15] Vladimir A. Gurvich, Alexander V. Karzanov, and Leonid G. Khachiyan. Cyclic games and an algorithm to find minimax cycle means in directed graphs. *USSR Computational Mathematics and Mathematical Physics*, 28(5):85–91, 1988. DOI: [10.1016/0041-5553\(88\)90012-2](https://doi.org/10.1016/0041-5553(88)90012-2).
- [16] Marcin Jurdziński. Deciding the winner in parity games is in $UP \cap co-UP$. *Information Processing Letters*, 68(3):119–124, 1998. DOI: [10.1016/S0020-0190\(98\)00150-1](https://doi.org/10.1016/S0020-0190(98)00150-1).
- [17] Marcin Jurdziński, Rémi Morvan, Pierre Ohlmann, and K. S. Thejaswini. A symmetric attractor-decomposition lifting algorithm for parity games. arXiv:2010.08288v1, 2020.
- [18] Alexander V. Karzanov and Vasilij N. Lebedev. Cyclical games with prohibitions. *Mathematical Programming*, 60:277–293, 1993. DOI: [10.1007/BF01580616](https://doi.org/10.1007/BF01580616).
- [19] Alexander Kozachinskiy. Polyhedral value iteration for discounted games and energy games. In Dániel Marx, editor, *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 600–616. SIAM, 2021. DOI: [10.1137/1.9781611976465.37](https://doi.org/10.1137/1.9781611976465.37).
- [20] Karoliina Lehtinen, Paweł Parys, Sven Schewe, and Dominik Wojtczak. A recursive approach to solving parity games in quasipolynomial time. *Logical Methods in Computer Science*, 18(1):8:1–8:18, 2022. DOI: [10.46298/LMCS-18\(1:8\)2022](https://doi.org/10.46298/LMCS-18(1:8)2022).
- [21] Yury M. Lifshits and Dmitri S. Pavlov. Potential theory for mean payoff games. *Journal of Mathematical Sciences*, 145(3):4967–4974, 2007. DOI: [10.1007/s10958-007-0331-y](https://doi.org/10.1007/s10958-007-0331-y).
- [22] Bruno Loff and Mateusz Skomra. Smoothed analysis of deterministic discounted and mean-payoff games. In Karl Bringmann, Martin Grohe, Gabriele Puppis, and Ola Svensson, editors, *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024)*, volume 297 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 147:1–147:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. DOI: [10.4230/LIPIcs.ICALP.2024.147](https://doi.org/10.4230/LIPIcs.ICALP.2024.147).
- [23] Pierre Ohlmann. A symmetric recursive algorithm for mean-payoff games. arXiv:2603.07555v1, 8 March 2026. DOI: [10.48550/arXiv.2603.07555](https://doi.org/10.48550/arXiv.2603.07555).

- [24] OpenAI. Randomized quasipolynomial-time mean-payoff games. OpenAI Math Release preprint [OAI:Randomized-quasipolynomial-time-mean-payoff-games-September-25-2026](#), 2026.
- [25] Paweł Parys. Parity games: Zielonka’s algorithm in quasi-polynomial time. In Peter Rossmanith, Pinar Hegghernes, and Joost-Pieter Katoen, editors, *44th International Symposium on Mathematical Foundations of Computer Science (MFCS 2019)*, volume 138 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 10:1–10:13. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019. DOI: [10.4230/LIPIcs.MFCS.2019.10](#).
- [26] K. S. Thejaswini, Pierre Ohlmann, and Marcin Jurdziński. A technique to speed up symmetric attractor-based algorithms for parity games. In Anuj Dawar and Venkatesan Guruswami, editors, *42nd IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2022)*, volume 250 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 44:1–44:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. DOI: [10.4230/LIPIcs.FSTTCS.2022.44](#).
- [27] Laurent Truffet. Looking for all solutions of a set of max-atoms solves the max atom problem in strongly polynomial time. *Modélisation des Systèmes Réactifs (MSR 2025)*, 2025. HAL: [hal-05491586](#).
- [28] Laurent Truffet. Substitution for minimizing/maximizing a tropical linear (fractional) programming. [arXiv:2603.26423v4](#), August 29, 2026. [arXiv:2603.26423v4](#).
- [29] Uri Zwick. Improved subexponential analysis of the Random-Action-Removal algorithm for 2-player turn-based games and non-binary AUSOs. [arXiv:2607.06334v1](#), 2026. DOI: [10.48550/arXiv.2607.06334](#).
- [30] Uri Zwick and Mike Paterson. The complexity of mean payoff games on graphs. *Theoretical Computer Science*, 158(1–2):343–359, 1996. DOI: [10.1016/0304-3975\(95\)00188-3](#).